

ASTRA AI Course Builder

Branch: `feature/ai-course-builder`

Feature summary

29 Jul 2026

This document explains **all changes** made for the AI Course Builder feature, **how they were implemented**, and how teachers and students use the flow.

1. Goal

Let a teacher upload a syllabus, let AI generate a roadmap, flashcards, quiz, and assessments, then require teacher review/publish before students can see that content.

```
Teacher uploads syllabus
→ AI generates draft artifacts
→ Teacher reviews / edits
→ Teacher publishes selected artifacts
→ Students see only published content
```

2. AI stack chosen

Choice	Why
Google Gemini via <code>google-gemai</code>	Free developer tier, file/PDF support, structured JSON output
Provider-agnostic interface	Can swap to OpenAI / Claude / local models later without rewriting routes
Mock provider fallback	Works without an API key during local development
Teacher-review publish model	AI drafts never go directly to students

Config keys added: `AI_PROVIDER`, `GEMINI_API_KEY`, `GEMINI_MODEL`, `UPLOADS_DIR`.

3. Branch and config setup

1. Created git branch `feature/ai-course-builder` from `main`.
2. Added `google-gemai` to `backend/requirements.txt`.
3. Extended `backend/.env.example` and `backend/app/core/config.py` with AI settings.

4. New backend files

File	Purpose
<code>backend/app/models/course_builder.py</code>	DB models: <code>CourseBuildJob</code> , <code>CourseArtifact</code>

backend/app/schemas/course_builder.py	Pydantic schemas for jobs, artifacts, roadmap/flashcards/quiz/assessment
backend/app/services/ai/provider.py	Abstract AI provider interface
backend/app/services/ai/gemini_provider.py	Gemini structured generation
backend/app/services/ai/mock_provider.py	Deterministic draft content when no Gemini key is set
backend/app/services/course_builder.py	Background job orchestration and artifact persistence
backend/app/api/routes/course_builder.py	Upload / generate / poll / list / publish API routes

5. How the backend works

5.1 Data model

- **CourseBuildJob** — tracks generation status (`PENDING` , `RUNNING` , `COMPLETED` , `FAILED`), requested artifact types, syllabus snapshot, errors.
- **CourseArtifact** — stores each generated piece (roadmap, flashcards, quiz, assessment) as JSON, with `is_published` defaulting to `false` .

5.2 API endpoints

Method	Path	Who	What it does
POST	/subjects/{id}/course-builder/upload-syllabus	Teacher	Uploads syllabus file; stores path; extracts text for txt/md
POST	/subjects/{id}/course-builder/generate	Teacher	Creates a job and runs generation in a background task
GET	/subjects/{id}/course-builder/jobs/{job_id}	Teacher	Polls job status
GET	/subjects/{id}/course-builder/artifacts	Teacher / Student	Teachers see drafts + published; students see published only
PATCH	/course-builder/artifacts/{artifact_id}	Teacher	Edit title/content and toggle publish

5.3 Permissions

- Edit/generate/publish: `SUPER_ADMIN` , class teacher of the classroom, or subject teacher of that subject.
- Students must be enrolled and the subject must be published; they only receive artifacts with `is_published=true` .

5.4 Generation pipeline

1. Teacher ensures syllabus text exists or uploads a file.

2. API creates a `CourseBuildJob` and returns `202 Accepted`.
3. Background task calls Gemini (if key present) or Mock provider.
4. Provider returns structured JSON matching Pydantic schemas.
5. Service saves one draft artifact per requested type.
6. Teacher edits and publishes selected artifacts.

6. Frontend changes

File	Change
<code>frontend/src/api/client.ts</code>	Skip forcing JSON Content-Type for <code>FormData</code> uploads
<code>frontend/src/api/types.ts</code>	Added <code>CourseBuildJob</code> , <code>CourseArtifact</code> , <code>artifact/status</code> types
<code>frontend/src/api/index.ts</code>	Added <code>courseBuilderApi</code> helpers
<code>frontend/src/pages/SubjectsPage.tsx</code>	Teacher builder UI + student published-content view inside subject workspace

6.1 Teacher UI

- Upload syllabus file (.txt / .md / .pdf)
- Generate AI drafts (roadmap, flashcards, quiz, assessments)
- Poll job status while generation runs
- Edit artifact JSON and title
- Publish / unpublish each artifact

6.2 Student UI

- Same Subjects page, "AI course content" section
- Shows only published artifacts
- Read-only preview of structured content

7. How this was executed

1. **Planned first** — selected Gemini + teacher-review publish model.
2. **Branched** — `feature/ai-course-builder`.
3. **Backend-first** — `config` → `models/schemas` → AI services → routes → mount in `main.py`.
4. **Frontend next** — `API client/types` → Subjects page teacher + student UX.
5. **Validated** —
 - `python -m compileall app`
 - `npm run build`
 - API flow: teacher generated 4 drafts, published 1, student saw only that 1

Validation result: Teacher created 4 draft artifacts; after publishing “AI Assessments”, student visibility count was 1 (published only). Mock provider was used when no Gemini key was configured.

8. Files touched (summary)

Modified

- backend/.env.example
- backend/requirements.txt
- backend/app/core/config.py
- backend/app/main.py
- backend/app/models/__init__.py
- frontend/src/api/client.ts
- frontend/src/api/index.ts
- frontend/src/api/types.ts
- frontend/src/pages/SubjectsPage.tsx

Added

- backend/app/api/routes/course_builder.py
- backend/app/models/course_builder.py
- backend/app/schemas/course_builder.py
- backend/app/services/ (AI providers + course builder orchestration)

9. How to use it

1. Start backend and frontend.
2. Optional: set `GEMINI_API_KEY` in `backend/.env`. Without it, mock drafts are generated.
3. Log in as class/subject teacher.
4. Open **Subjects** → select subject → save or upload syllabus.
5. Click **Generate AI drafts**.
6. Review/edit drafts → **Publish** selected ones.
7. Log in as enrolled student → open the published subject → view AI course content.

Runtime note: During validation, a stale process briefly blocked port 8000. A clean backend on port 8001 was used successfully for API verification. If the frontend proxy (port 5173 → 8000) fails, restart the backend cleanly on 8000 or temporarily point the proxy/API URL to 8001.

10. What was intentionally not built yet

- Student quiz attempts / scoring
- Assessment submissions
- Adaptive recommendations based on progress

- Redis/Celery queue (MVP uses FastAPI background tasks)

Those can be added next on the same branch without changing the draft → review → publish model.

Generated from the implemented work on branch `feature/ai-course-builder` .